

Introducción a la Ingeniería de Software:

Software: conjunto de programas y la documentación que los acompaña. Existen 3 tipos:

- System software: controlar los recursos de hardware de la PC
- Utilitarios: herramientas adicionales para mejorar la funcionalidad del OS
- Sistemas de aplicación: tareas específicas en la PC

El software se puede entender como conocimiento representado en distintos niveles de abstracción:

- La idea
- El diseño y la documentación
- El código
- Las bibliotecas, dependencias y el OS
- El código binario
- El hardware que corre el código

Etapas del software:

- Requerimientos: documento de especificación
- Prototipos: MVPs, prototipos digitales interactivos
- Diseños: diagramas, arquitectura, modelos de diseño
- Código: código fuente
- Despliegue: testing, CI/CD, llevarlo al ambiente de producción
- Producción

El software no es manufactura:

- El software es menos predecible
- No hay producción en masa: casi ningún software es igual a otro
- No todas las fallas son errores
- El software no se desgasta
- El software no es un producto físico (no está gobernado por las leyes físicas)

Las 4 Propiedades Irreducibles del Software Complejo:

- **Complejidad:** escalar software aumenta los elementos interactuando de forma no lineal. No hay piezas repetidas a nivel macro.
- **Conformidad:** debe adaptarse a interfaces humanas arbitrarias y sistemas sin principios unificadores, dictados por el entorno.
- **Mutabilidad:** es infinitamente maleable y constantemente sometido a la presión del cambio continuo.
- **Invisibilidad:** no tiene representación espacial o geométrica, no se puede hacer una abstracción visual.

Modelo Aristotélico del Software: diferencia dos dificultades a la hora de escribir código:

- El accidente (la representación): dificultades que acompañan la producción pero no son inherentes, como la sintaxis complicada, recursos computacionales limitados, altos tiempos de compilación, etc.

- La esencia (el concepto): la construcción de estructuras conceptuales entrelazadas, como algoritmos, flujos de datos, relaciones, etc.

El modelo plantea que escribir código no es el problema real del desarrollo, ya que los problemas de representación están casi resueltos. La mayoría de los avances en la historia del software fueron para resolver lo accidental, como lenguajes de alto nivel con una sintaxis más amigable o los IDE que facilitan la conexión de programas y formatos.

Lo difícil es la especificación, diseño y validación de la estructura conceptual. Las 4 propiedades del software complejo son problemas de la esencia.

Esto es el cuello de botella humano en el desarrollo con IA, no decidir el cómo hacerlo sino decidir el qué hacer ante una serie de ideas, requerimientos y reglas de negocio.

Cómo Abordar la Esencia: la IA es muy buena para la representación, pero la esencia sigue requiriendo intervención humana.

- **Buy vs Build:** evitar construir todo por completo. La integración es más barata que generar código nuevo.
- **Rapid Prototyping:** extraer requerimientos iterativamente.
- **Grow, not Build:** desarrollo incremental. Hacer un esqueleto mínimo e ir engrosando el sistema manteniendo un sistema funcional.
- **Grandes Diseñadores:** el diseño sigue siendo un acto humano. La diferencia en un gran diseño sigue siendo de buenos diseñadores, no de una solución automática o genérica (bala de plata).

Ingeniería de Software:

"La ingeniería de software es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software". IEEE.

"La ingeniería de software es una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde la concepción inicial del sistema hasta su operación y mantenimiento". Sommerville.

Cuerpo de Conocimiento de la Ingeniería de Software (SWEBoK): 15 áreas de conocimiento que abarca la IS.

- Requerimientos de software
- Diseño de software
- Construcción de software
- Testing the software
- Mantenimiento de software
- Administración de configuración de software
- Administración de ingeniería de software
- Proceso de ingeniería de software
- Modelos y métodos de ingeniería de software
- Calidad de software
- Prácticas profesionales de ingeniería de software
- Economía de ingeniería de software
- Fundamentos de la computación
- Fundamentos matemáticos
- Fundamentos de la ingeniería

Cuerpo de Conocimiento de la Ingeniería de Sistemas (SEBoK): partes -> áreas de conocimiento:

- **Fundamentos de la Ingeniería en Sistemas**
 - Fundamentos de sistemas
 - Enfoque de sistemas aplicados a sistemas de ingeniería
 - Ciencia de sistemas
 - Pensamiento sistémico
 - Representación de sistemas con modelos
- **Ingeniería de Sistemas y Gestión:**
 - Introducción a los procesos del ciclo de vida
 - Modelos de ciclo de vida
 - Definición del concepto
 - Definición del sistema
 - Realización del sistema
 - Despliegue y uso del sistema
 - Gestión de la ingeniería de sistemas
 - Gestión de vida del producto y el servicio
 - Estándares de ingeniería en sistemas
- **Aplicaciones de la Ingeniería de Sistemas:**
 - Ingeniería de sistemas de producto
 - Ingeniería de sistemas de servicio
 - Ingeniería de sistemas de empresa
 - Sistema de sistemas
 - Ingeniería de sistemas de salud
- **Facilitadores de la Ingeniería de Sistemas:**
 - Facilitando empresas y negocios
 - Facilitando equipos
 - Facilitando individuos
- **Disciplinas Relacionadas:**
 - Ingeniería de software
 - Administración de proyectos
 - Ingeniería industrial
 - Ingeniería de especialidad

Disciplinas a ver en la materia:

- **Técnicas:**
 - Prueba
 - Despliegue
- **Gestión:**
 - Planificación de proyecto
 - Monitoreo y control de proyectos
- **Soporte:**
 - Gestión de configuración de software
 - Aseguramiento de calidad
 - Métricas

Gestión de Proyectos:

Proceso de Software: conjunto estructurado de actividades para desarrollar un sistema de software que a partir de un conjunto de entradas producen una salida. Las actividades van a variar dependiendo de la organización y el tipo de software a desarrollar. Debe ser explícitamente modelado si va a ser administrado. Pueden ser:

- **Definidos:** están inspirados en las líneas de producción. Asumen que podemos repetir el mismo proceso una y otra vez y obtener los mismos resultados. La administración y el control provienen de la predictibilidad. (Nota: no significa que repitiendo las actividades obtenemos lo mismo, ya que ningún software es igual a otro. Significa que aplicando el mismo proceso, con las mismas etapas, en cualquier contexto, obtenemos resultados igual de satisfactorios).
- **Empíricos:** asume procesos complicados con variables cambiantes, lo que implica que repitiendo el proceso se pueden obtener resultados distintos, por lo que se basa en la experiencia. La administración y control es a través de inspecciones frecuentes y adaptaciones. Trabajan bien con procesos creativos y complejos. Los pilares son:
 - Inspección: detectar variaciones y emergencias.
 - Adaptación: ajuste inmediato a variaciones.
 - Transparencia: revelar la realidad tal como es.

Estructura de un proceso definido: secuencia de pasos con entregables o hitos bien definidos en cada paso.

Patrón de conocimiento en procesos empíricos:

- Asumir (tener una hipótesis)
- Construir
- Retroalimentar (tomar información)
- Revisar (comparamos la hipótesis con el resultado)
- Adaptar (y vuelta al principio)

Proceso-Proyecto-Producto: un proceso es una plantilla, un conjunto de pasos a seguir para obtener una salida de una entrada. Se automatiza con herramientas. Un proyecto es una instancia del proceso, donde se involucran personas para obtener un producto. El producto es la salida del proceso, es el software construido.

Características de los Proyectos:

- **Orientados a objetivos:** están dirigidos a obtener resultados y eso se refleja con objetivos. Los objetivos guían al proyecto y deben ser claros y alcanzables,
- **Son únicos:** pueden ser similares pero tienen características que los diferencian.
- **Duración limitada:** son limitados y terminan cuando se alcanza el objetivo.
- **Tienen tareas relacionadas entre sí:** están compuestos por tareas complejas basadas en esfuerzos y recursos.

Ciclo de Vida: serie de pasos a través de los cuales el producto o proyecto progresa. Es una abstracción de todas las tareas que se realizan en el proyecto o producto.

El ciclo de vida del proyecto es más corto que el del producto. Cada actualización que requiere el producto es el inicio de un nuevo proyecto, y su fin es una nueva versión del producto.

Ciclo de Vida de un Proyecto: es una representación de un proceso que grafica su descripción desde una perspectiva particular. Especifica:

- Fases del proceso
- Orden de las fases

Clasificaciones:

- **Secuencial:** ejecutar una fase después de otra generalmente sin retorno entre ellas. Algunos, como el ciclo en cascada, puede devolver información.
- **Iterativo/incremental:** el ciclo se divide en tareas que se ejecutan de forma cíclica. En cada ciclo se agregan algunas funcionalidades. El producto terminado se obtiene luego de varios ciclos. Cada iteración provee retroalimentación a la siguiente y entrega una versión parcial pero funcional del producto.
- **Recursivo:** todas las definiciones dicen lo mismo que iterativo e incremental (lo llaman iterativo y creciente). Según resumen: para proyectos de alto riesgo.

Ley de Brook: agregar personas a un proyecto retrasado lo retrasa más. Nuevas personas significa que se gasta tiempo en enseñarles el proyecto y hay que redistribuir el trabajo. También con más personas se vuelve más difícil la comunicación, que puede hacer fallar un proyecto.

Gestión de Proyectos: la administración de proyectos es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer sus requerimientos. Incluye:

- Identificar requerimientos
- Establecer objetivos claros y alcanzables
- Adaptar especificaciones, planes y enfoques a los intereses de los stakeholders

Restricción Triple: hay que balancear 3 variables que dependen entre sí:

- Costo
- Tiempo
- Alcance

Líder de Proyecto (PM): buscar en el resumen, me confundí y puse PO

Equipo de Proyecto: grupo de personas comprometidas en alcanzar un conjunto de objetivos. Características:

- Diversos conocimientos y habilidades
- Trabajan juntos efectivamente
- Usualmente es un grupo pequeño
- Tienen sentido de responsabilidad compartido

Plan de Proyecto: es una hoja de ruta del proyecto. Documenta

- ¿Qué hacemos?
- ¿Cuándo lo hacemos?
- ¿Cómo lo hacemos?

- ¿Quiénes lo hacen?

Planificación:

- **Definir alcance:**
 - Alcance del producto: todas las características de un producto o servicio. Se mide contra la especificación de requerimientos.
 - Alcance del proyecto: todo el trabajo que hay que hacer para entregar el producto con todas las funcionalidades. Se mide contra el plan del proyecto.
- **Definir un Proceso y Ciclo de Vida:** elegir el proceso y el ciclo de vida a utilizar.
- **Estimar:**
 - Tamaño
 - Esfuerzo
 - Calendario
 - Costo
 - Recursos críticos
- **Gestión de Riesgos:** identificar los riesgos (algo que puede suceder o no pero afecta al éxito del proyecto) más probables o con mayor impacto y definir acciones para evitarlos o mitigar sus efectos.
- **Asignación de Recursos**
- **Programación de Proyectos**
- **Definición de Métricas:** medidas que permiten ver si el proyecto está desviado del plan o no. Pueden ser de:
 - Proceso (organización): permiten saber si la organización como tal, independientemente de un proyecto, está trabajando bien. Ej: porcentaje de proyectos que terminan a tiempo.
 - Proyecto (equipo de desarrollo): permiten saber si un proyecto está siguiendo lo planificado. Ej: comparación del tiempo calendario con el real.
 - Producto (desarrollador): miden características del producto. Ej: métricas de defectos, porcentaje de cobertura con unit tests.
- **Definición de Controles:** determinar cómo se va a verificar que el proyecto esté bien encaminado.

Gestión de Producto:

Arquitectura del Valor: asegurarse de que el producto entrega valor a los usuarios. Abarca desde validar una hipótesis de un producto de software a generar lealtad en los usuarios.

El mayor riesgo en el desarrollo no es el fracaso técnico, es que el producto no sea utilizado. Escribir código no es sinónimo de valor. Hasta el 45% de las funcionalidades de un producto nunca son utilizadas. El verdadero valor está en la adopción. El rol del ingeniero ahora no es estar focalizado en tareas, sino en experiencias para el usuario.

Ciclo Lean (la audacia del cero): no esperar a tener asegurado el éxito y lanzar un producto perfecto para no mostrar una versión parcial o con poca adopción (audacia del cero). Eso solo genera aislamiento y aumenta el riesgo del rechazo masivo. Es mejor una iteración continua para validar la hipótesis más temprano y ajustar el producto a las necesidades de los usuarios.

Validación Temprana: permite validar una idea antes de construirla por completo, mitigando el riesgo de baja adopción masiva al terminarlo. Estrategias:

- Hipótesis de valor: ¿El producto le entrega valor al usuario?
- Hipótesis de crecimiento: ¿Cómo descubren el producto nuevos usuarios?
- Prototipo: producto con características faltantes.
- Prueba de humo: mostrar funcionalidad sin tener nada construido.

MVP: un experimento diseñado para validar la hipótesis lo antes posible. No es solo el diseño técnico, debe proveer una muestra de lo que será el producto final. Tiene que enfocarse en la propuesta de valor única (UVP) y ofrecer lo mínimo indispensable para ser viable. Ej: no enfocarse solo en todas las funcionalidades, incluir sólo las indispensables y mejorar usabilidad y fiabilidad sobre esas.

UVP: la hipótesis principal, es lo que hace que el producto sea único en el mercado. Si al lanzar un MVP cada usuario pide algo distinto, no se encontró una buena UVP. Si en cambio todos piden lo mismo se ha encontrado una UVP potencial y hay que validarla con un nuevo MVP.

Matriz de Priorización: decide qué entra al MVP.

	Importante	No importante
Urgente	1. Componentes críticos de la UVP	3. Integración con APIs de terceros
No urgente	2. Funcionalidades de maduración para la beta.	4. Sacar cosas no importantes ni urgentes

MMF (Minimum Marketable Feature): la pieza más pequeña de funcionalidad que aporta valor inmediato y crecimiento, y por lo tanto puede comercializarse y ser rentable. Alta certeza comercial.

MVF (Minimum Viable Feature): una función pionera para probar un área nueva en un producto, como un mini MVP dentro de un producto ya existente. Alta incertidumbre.

Una vez encontrada la UVP se construye el MVP combinando MMF para explotar certezas y MVF para explorar nuevas áreas.

Cronología de Lanzamiento:

1. MVP: validar idea. No es un producto final.
2. MMP (Minimum Marketable Product): primer release dirigido a early adopters. Es el MMR 1. Contiene los MMF esenciales.
3. MMR N (Minimum Marketable Release): el incremento más pequeño que le ofrece valor a los usuarios. MMR N-1 + MMF. Cada MMR agrega características al producto respecto al anterior (incrementos sucesivos).

MLP (Minimum Lovable Product): en mercados maduros o con competidores bien financiados, un MVP no es suficiente y puede parecer un intento de lucro a corto plazo. El MLP permite:

- Diferenciación: destacar en un mercado maduro y con competidores bien financiados.
- Reputación de marca: mostrar interés por las necesidades de tus usuarios.
- Feedback superior: crear conexión emocional temprana para retener usuarios reales y tener flujos de retroalimentación.

	MVP	MMP	MLP
Objetivo	Validar concepto	Atender base amplia de usuarios	Crear conexión emocional
Enfoque de diseño	Investigación	Lanzamiento al mercado	Impacto emocional
Resultado	Informar alcance del producto	Atracción masiva del mercado	Generar lealtad temprana y retención

Ágil:

Filosofía Ágil: movimiento que define una serie de principios a seguir que guían el desarrollo del producto, buscando el equilibrio entre adherirse a un proceso y trabajar eficientemente. Enfatiza la flexibilidad y adaptación, la transparencia y la inspección; balanceando ningún proceso con el exceso de procesos. Se sustenta en procesos empíricos y ciclos de vida iterativos.

- Adaptables en vez de predictivos
- Orientados a personas en vez de procesos

No es un proceso, es una filosofía.

Tiene 4 valores y 12 principios:

Valores:

- **Individuos e Interacciones sobre Procesos y Herramientas**
- **Software Funcionando sobre Documentación Exhaustiva**
- **Colaboración con el Cliente sobre Negociación Contractual**
- **Responder al Cambio sobre Seguir un Plan**

Principios:

1. La mayor prioridad es satisfacer al cliente
2. Aceptar que los requisitos cambien
3. Entregar software funcional frecuentemente
4. Distintos perfiles deben trabajar juntos
5. Desarrollar proyectos con individuos motivados
6. La mejor comunicación es cara a cara
7. La principal medida de éxito es el software funcionando
8. Los procesos ágiles promueven el desarrollo sostenible

9. La atención continua a la excelencia y el buen diseño mejora la agilidad
10. La simplicidad es esencial
11. Las mejores arquitecturas y diseños surgen de equipos auto organizados
12. A intervalos regulares el equipo reflexiona cómo ser más efectivo y ajusta su comportamiento

Valores de Equipos Ágiles:

- Planificación continua multinivel
- Equipos completos auto organizados
- Entregas frecuentes e iterativas
- Prácticas de ingeniería disciplinadas
- Integración continua
- Testing concurrente

Requerimientos Ágiles:

- Usar el valor para construir el producto correcto
- Usar historias y modelos para mostrar qué construir
- Determinar que es solo lo suficiente

Los requerimientos cambiantes pueden ser una ventaja competitiva si se trabaja sobre ellos. En cada iteración se completan los requisitos prioritarios. Los requisitos nuevos se priorizan y agregan a la pila. Los requisitos pueden priorizarse y eliminarse en cualquier momento. El valor es la obtención de beneficio tangible o intangible, es la utilidad, beneficio o satisfacción que se le da a los usuarios finales.

Tipos de Requerimientos:

- **Requerimiento de Negocio:** organizacional.
- **Requerimiento de Usuario:** que hace el usuario.
- **Requerimiento Funcional**
- **Requerimiento no Funcional**
- **Requerimiento de Implementación**

Los primeros dos son en el dominio del problema, los demás en el de la solución (requerimientos de software).

La Triple Restricción en Ágil: en los modelos tradicionales se establece un plan rígido a seguir durante todo el proyecto. El alcance queda fijo y varía el tiempo y el costo ante un cambio de las variables. Esto se denomina el triángulo de hierro. El triángulo ágil en cambio tiene tiempo (duración de cada iteración) y costo (cantidad de personas) fijos y varía el alcance en cada iteración y según el plan se vaya adaptando a los requerimientos.

Principios Ágiles Relacionados a Requerimientos:

- 1
- 2
- 4
- 6
- 11

User Story: técnica para capturar requerimientos. Son verticales, esto es que una US afecta varias capas de la aplicación y puede dar un incremento de valor por sí sola (a diferencia de algo horizontal que cubre una sola capa y necesita varias porciones para cubrir toda la arquitectura y agregar valor). Son multipropósito, son:

- Necesidades del usuario
- Descripción del producto
- Ítems de planificación
- Tokens para conversaciones
- Mecanismos para diferir una conversación

Está formada por:

- **Tarjeta:** contenedor de la descripción de la US, la conversación y los criterios de aceptación.
- **Conversación:** Como <rol> quiero <actividad> para <valor de negocio que le aporta>
- **Confirmación:** criterios de aceptación. Todas las acciones que el usuario debe o puede poder hacer y las condiciones que se tienen que cumplir para considerar la US implementada. Es lo que debe cumplirse para que genere valor. Definen los límites de la US.
 - Buenos criterios de aceptación:
 - Definen intención además de la solución
 - Son independientes de la implementación
 - Relativamente de alto nivel (los detalles van en documentación interna del equipo o pruebas automatizadas)

No son especificaciones detalladas de los requerimientos (como CU), son expresiones de intención. No se detallan al principio del proyecto para evitar especificaciones anticipadas y definiciones limitadas de la solución. Necesitan poco mantenimiento y pueden ser descartadas. Junto con el código son la entrada para la documentación posterior.

Modelado de Roles: identificar el perfil de las personas que usarán el sistema.

- **Tarjeta de Rol de Usuario:** una descripción del puesto que ocupa ese rol en la organización, el conocimiento que tiene, el uso que le dará al sistema y lo que le es más importante.
- **Personas:** modelar el rol describiendo a una persona concreta que lo ocupa, incluyendo sus datos.
- **Personas Extremas:** modelar el rol listando algunos ejemplos extremos de personas que ocupan ese rol con necesidades muy variadas, para captar los extremos.
- **Usuarios Representantes (proxies):** descripción genérica
 - Gerente de usuarios
 - Gerente de desarrollo
 - Alguien de marketing
 - Clientes
 - Etc

Pruebas de Aceptación de US: expresan detalles de la conversación y complementan la US. La prueba debe declarar las condiciones preexistentes de la prueba, las acciones a tomar y el resultado esperado (pasa o falla)

Definición de Listo o Criterio de Ready: es lo que define cuando la US está lista para implementarse. Se define por el equipo.

Modelo Invest: modelo para definir US listas.

- Independiente: calendarizable e implementable en cualquier orden.
- Negociable: el qué, no el cómo, ya que la US no tiene detalles.
- Valiosa: debe tener valor para el cliente.
- Estimable: poder definir el esfuerzo que va a tomar.
- Small: poder ser consumida en una iteración
- Testeable: demostrar que fue implementada

Tema: conjunto de US relacionadas.

Épica: una historia de usuario grande.

Spike: tipo especial de US que se usa para quitar incertidumbre de otra US o requerimiento.

- **Técnicas:** utilizadas para investigar enfoques técnicos de la solución, como investigar una herramienta a utilizar.
- **Funcionales:** se usan cuando hay dudas de cómo los usuarios interactúan con el sistema. Pueden ser crear prototipos por ejemplo.

Generalmente van en la iteración anterior a la US sobre la que hay dudas.

Deben ser la excepción, ya que siempre habrá cierta incertidumbre.

Estimaciones

Estimar: predecir en un momento en el que hay incertidumbre. Por definición no es precisa. Estimar no es planear, pero es la base de los planes. Los planes pueden ser distintos a la estimación, pero a mayor diferencia mayor riesgo.

El objetivo es reducir la incertidumbre. A medida que avance el proyecto las estimaciones se volverán más precisas, por lo que las estimaciones más realistas no son las que se hacen en las primeras iteraciones.

Técnicas de Estimación:

- **Contar:**
 - Funcionalidades
 - Requerimientos
 - US
 - CU
 - etc.

Métodos:

- **Basados en la Experiencia:** se basa en la experiencia pasada en proyectos similares para estimar.
 - **Datos Históricos:** comparar el proyecto nuevo con datos de uno anterior, específicamente tamaño, tiempo, esfuerzo y defectos. El problema es que los dominios pueden ser muy distintos.

- **Juicio Experto:** un experto con experiencia en proyectos similares evalúa el nuevo proyecto y da su estimación subjetiva. Se basa en sus conocimientos y experiencias.
 - Puro: se le pregunta a un experto. Problema: depende todo de una persona.
 - Delphi: se reúne un grupo de expertos, debaten entre ellos sobre las especificaciones y luego de forma anónima hacen su estimación. Cada uno evalúa las estimaciones y luego repiten el proceso hasta que convergan. Problema: lleva más tiempo.
- **Analogía:** se basa en datos históricos pero se seleccionan específicamente proyectos similares.
- **Basados Exclusivamente en Recursos:** se basa en la cantidad y tipo de recursos necesarios y se estima con su disponibilidad y costo.
- **Basados Exclusivamente en el Mercado:** se basa en el costo de proyectos similares en el mercado para determinar esfuerzo y tiempo.
- **Basados en los Componentes del Producto o el Proceso:** se descompone el producto en componentes y se estima cada componente por separado, y luego se suman.
- **Métodos Algorítmicos:** se basa en modelos matemáticos y estadísticos.

Estimaciones Ágiles: se estima usando una medida relativa llamada Story Point, que mide complejidad, esfuerzo y riesgo, que no está basada en el tiempo. Se basa en que:

- Las personas no saben estimar en términos absolutos
- Somo buenos comparando
- Comparar es más rápido
- Se obtiene una mejor dinámica grupal
- Se usa más tiempo en analizar la US

El tamaño indica la cantidad de trabajo para hacer esa US. Indica

- La complejidad
- El trabajo requerido
- El tamaño de la US

El tamaño no es esfuerzo.

Escalas: cómo medir el tamaño de distintas US. Una vez elegida no se puede cambiar.

- Lineal
- Talles (XS, S, M, L, XL)
- Potencias de 2
- Fibonacci

Velocidad: medida (métrica) del progreso de un equipo. Se completa sumando el número de SP de las US que el equipo completó en esa iteración. No se cuentan US parciales. La velocidad se usa para corregir la estimación.

Métodos para Estimar - Poker Planning: se usa en ambientes ágiles. Participan los desarrolladores y usan la escala de Fibonacci.

Primero se elige una US canónica que se usa como base para comparar las demás. La US canónica debe ser la más básica y se le da un valor de 1 SP. Luego, por cada US los participantes discuten y hacen preguntas al PO, y luego cada uno elige un valor de la serie para asignarle al mismo tiempo. Después discuten sobre las diferencias, en particular sobre los valores más altos y bajos. Luego repiten hasta que todos coincidan y pasan a la siguiente.

Combina la estimación por analogía y el juicio experto Delphi. Las US grandes se deben tratar de dividir para cumplir el criterio de ready.

Escala de referencia:

- 1-2: funcionalidad pequeña o cosmética.
- 2-3: pequeña a mediana. Lo ideal.
- 5: mediana. Lo ideal.
- 8: una funcionalidad grande que puede hacerse en una iteración. Es aceptable pero debe evaluarse si se puede dividir.
- 13: debería ser dividida.
- 21: hay que replantearse la razón del negocio de tener una US tan grande.
- 34: es imposible hacerla en una iteración.

Scrum:

Scrum es un framework liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.

Principios:

- Desarrollo iterativo
- Control de proceso empírico
- Auto organización
- Colaboración
- Priorización basada en valor
- Time-boxing

Ciclo OODA (ciclo de empirismo): ciclo para responder rápidamente a los cambios del mercado y la incertidumbre.

- **Observar:** apertura y respeto. Reunir evidencia honesta y perspectivas diversas.
- **Orientar:** coraje. Interpretar la realidad, desafiar suposiciones y navegar la incertidumbre.
- **Decidir:** foco. Seleccionar experimentos seguros y priorizar el backlog.
- **Actuar:** compromiso. Ejecutar hacia el spring goal dentro de la restricción de tiempo.

Valores SCRUM:

- Coraje
- Foco
- Compromiso
- Respeto
- Apertura

Ecosistema de SCRUM:

- **Scrum Team:** es la unidad central de SCRUM. Debe funcionar como una célula profesional.
 - **Autogestión:** el equipo decide qué, cómo, cuándo y dónde sin microgestión externa.
 - **Profesionalismo:** responsabilidad total y excelencia técnica y ética.
 - **Diversidad Cognitiva:** personas con roles y habilidades variadas en el equipo.
- **Supporters:** líderes y gerentes que apoyan activamente la adopción de SCRUM. Eliminan barreras organizacionales, proveen recursos y cobertura política. No todos los gerentes son supporters, solo aquellos que apoyan la adopción.
- **Stakeholders:** clientes, usuarios, decisores e incluso entidades no humanas. Proveen feedback, requisitos y validan el valor.

Roles:

- **Product Owner:**
 - Tipo: humano.
 - Responsabilidad: maximizar el valor a largo plazo, gestionar el product backlog y la estrategia del producto.
- **Product Developer:**
 - Tipo: humano o automatizado.
 - Responsabilidad: crear incrementos utilizables, adherirse a la definición de salida temprana y autogestionar el plan del sprint.
- **Scrum Master:**
 - Tipo: humano.
 - Responsabilidad: guía la efectividad del equipo y la organización en la adopción de SCRUM. Es un agente del cambio.
- **Stakeholder:**
 - Tipo: humano o entidad.
 - Responsabilidad: entidades interesadas o afectadas por el producto.
- **Supporter:**
 - Tipo: stakeholder específico.
 - Responsabilidad: eliminar impedimentos organizacionales y fomentar un clima para la autogestión.

PO: el qué. Dev: el cómo. SM: la efectividad.

IA como Developer:

- Automatización y generación de incrementos.
- Análisis y detección de patrones (aumentación cognitiva).

Human in the Loop:

- La responsabilidad final siempre es humana.
- Uso ético y supervisado.
- Transparencia para reducir sesgos.

Liderazgo y Cambio:

- Liderazgo:
 - Es una conducta, no un cargo.
 - Crear claridad y un entorno seguro para la emergencia.

- Decisiones basadas en hechos reales, no reportes distantes.
- Cambio:
 - Es emergente.
 - No se puede forzar en una dirección.
 - Se guía una dirección del viaje.
 - Los líderes actúan como catalizadores.

Sprint: una iteración en el ciclo de vida del proyecto. Dura máximo 4 semanas. Se define un objetivo del sprint, el Sprint Goal, que es el resultado comercial y es inalterable. También se define el alcance (scope) que puede renegociarse entre el PO y los devs si ocurre aprendizaje.

El sprint actúa como un contenedor de consistencia en el que es seguro fallar. No hay sprints de prueba.

- Crea hábitos y reduce la complejidad cognitiva.
- Controla riesgo, sprints más cortos reducen el costo de un error.

Rituales:

- **Planning:** ocurre al principio del sprint. Define:
 - Por qué: el objetivo.
 - Qué: ítems del backlog.
 - Cómo: plan técnico de ejecución.
- **Daily:** re-planificación diaria de 15 minutos entre developers hacia el objetivo. Enfoque en el flujo y eliminación de bloqueos. No es un reporte de estado para el SM, es una sincronización táctica.
- **Review:** inspección del resultado y la salida al final del sprint. Es una sesión colaborativa. Evalúa si resolvieron el problema del usuario.
- **Retrospective:** mejorar el sistema en base a lo aprendido. Foco en la calidad del equipo, procesos y herramientas.

Timebox: tiempos máximos de cada ritual.

- **Sprint:** 1 mes.
- **Sprint Planning:** 8 horas máximo para un mes (2 horas por semana entonces?)
- **Daily Meetings:** 15 minutos.
- **Sprint Review:** 4 horas máximo para un mes (1 hora por semana?)
- **Sprint Retrospective:** 3 horas máximo para un mes.
- **Refinamiento del PB:** 10% de la duración del sprint.

Product vs Project Thinking: los proyectos terminan pero los productos evolucionan. El objetivo del producto debe guiar el proyecto. SCRUM gestiona el ciclo de vida completo. El descubrimiento y el desarrollo no son fases separadas, ocurren dentro del sprint.

Artefactos y Compromisos:

Artefacto	Compromiso Asociado
Product Backlog: lista emergente y ordenada de lo que se necesita.	Product Goal: objetivo estratégico a mediano y largo plazo (el por qué)

	estratégico)
Sprint Backlog: plan de los devs para el sprint.	Sprint Goal: objetivo único del sprint (el por qué táctico)
Incremento: el nuevo valor entregado.	Definition of Done: estándar de calidad requerido.

Product Backlog (PB): es una lista de lo que el producto necesita. Es un organismo vivo que se va refinando a medida que avanza el proyecto, no un plan fijo. Empieza como ideas difusas e hipótesis y se refina a requerimientos y US.

- Refinamiento continuo
- Aceptar la incertidumbre y validar todo
- Priorizar por valor

Definición de Hecho: tiene una doble definición con dos filosofías:

- **Output:** mide la calidad técnica. Responde a la pregunta de si se hizo de forma correcta (ej. código limpio, testeado, documentado). Se mide al terminar el sprint.
 - **Definición:** estándar formal y conocido.
 - **Contenido:** estándares técnicos (como CI/CD) y cualidades de uso (como accesibilidad). No son los criterios de aceptación de un ítem.
 - **No es negociable:** en un sprint se negocia alcance o fidelidad, nunca calidad.
 - **Responsabilidad colectiva:** todo el equipo se adhiere.
- **Outcome:** mide el valor de negocio aportado. Responde a la pregunta de si se hizo lo correcto (ej. cambio en el comportamiento del usuario, impacto en el negocio, validación de hipótesis). Se mide antes, durante y después del lanzamiento, por lo que es un lagging indicator. Se asocia con el producto entero.
 - **Definición previa:** se define antes de comenzar. El PO tiene la palabra final. Aplica al producto y/o al product goal.
 - **Evidencia:** favorecer la evidencia directa (como impacto en el negocio o usuario) sobre circunstanciales (como métricas internas de SCRUM).
 - **Medición:** uso del product telemetry. Se revisa en cualquier momento del sprint.
 - **Se guía por el outcome criteria de los ítems.**

El output es un requisito previo, el outcome es el éxito real.

Estado Ready: no es un compromiso como los otros dos, es una cualidad que surge del refinamiento continuo y de los otros dos. Reduce el riesgo y fomenta la calidad antes de construir. Se asocia a un ítem. Incluye:

- **Claridad absoluta:** ítem desglosado y sin ambigüedades. Incluye acceptance criteria (output) y outcome criteria.
- **Tamaño y Factibilidad:** se puede completar en pocos días. Dimensionado por los devs. Dependencias mapeadas y mitigadas.
- **Investigación Previa:** validación del problema y pruebas de concepto antes de comenzar.
- **Colaboración Activa:** interacción directa entre devs y stakeholders.
- **Coherencia del Rumbo:** sostener un sprint goal alineado con el product goal.

SCRUM 2026:

- **Roles:** añade el rol de supporter y declara al PO como obligatoriamente humano y responsable ético y estratégico.
- **Compromisos:** establece la diferencia entre definition of output done y output done.
- **Integración de IA:** describe la integración de la IA como un socio, potenciador y miembro del equipo; pero siempre con responsabilidad final humana.
- **Teoría y Pensamiento Adaptativo:** boludeces como producto y project thinking.
- **Prácticas Refinadas y Planificación:** inclusión del product vision como el objetivo a largo plazo y el product goal como un compromiso a mediano plazo.

Herramientas:

- **Taskboard:** post it con tareas para hacer. Deben ser granulares.
 - Código de ítem.
 - Nombre de la actividad.
 - Número de día del sprint.
 - Iniciales de la persona responsable.
 - Horas estimadas.
- **Sprint Burndown Chart:** gráfico con días del sprint en el eje x y puntos de historia en el eje y.
 - Línea ideal: desarrollo estimado de las US. Empieza en la sumatoria de los SP de todas las US y termina en 0. (tiene que ser lineal?)
 - Desarrollo real: función que muestra día a día los SP restantes. Puede o no llegar a 0 el último día. Si está por debajo de la línea estimada están adelantados, y si está encima atrasados.
 - SP finalizados: histograma de SP finalizados en cada día.
- **Gráfico de Seguimiento del Producto:** como el anterior pero con sprints en vez de días y la suma de todos los SP en vez de los de las US de ese sprint. Puede ser burndown (del total a 0, las líneas miden cuánto falta) o burnup (de 0 al scope, miden cuánto está hecho). Burnup es mejor porque permite cambiar el scope sin modificar los registros pasados.

Niveles de Planificación:

- **Estrategia**
- **Portfolio:** filtro económico
- **Producto:**
 - Entrada: idea.
 - Proceso: hacer backlog, roadmap, visión y adquirir conocimiento.
 - Participantes: stakeholders internos y equipo SCRUM.
 - Salida: visión del producto.
- **Release:**
 - Entrada: visión, backlog, roadmap, velocidad.
 - Proceso: revisar restricciones y MRFs, planear sprints.
 - Participantes: stakeholders internos y equipo SCRUM.
 - Salidas: release plan con rango de funcionalidades y costo fijo o rango de sprints y rango de costos, MRFs y mapa de sprints. Los releases pueden ser cada cierto grupo de sprints, cada sprint o cada funcionalidad añadida.

- **Iteración:** sprint planning.
 - Entrada: product backlog.
 - Proceso: refinarlo.
 - Salida: tareas.
- **Día:** ejecución del sprint.

Métricas: guiadas por la filosofía de solo testear lo necesario. La medición es una salida, no una actividad.

Principios ágiles:

- Satisfacer al cliente
- Software trabajando es la medida de progreso

Métricas:

- **Running Tested Features (RTF)**
- **Capacidad:** horas disponibles del equipo para un sprint. Días de trabajo de cada miembro * horas de trabajo cada día. También se puede medir en SP.
- **Velocidad**

Software Control Management

SCM: actividad de soporte transversal a todo el proyecto que aplica dirección y monitoreo administrativo y técnico para:

- Identificar y documentar características técnicas y funcionales de los ítems de configuración:
- Controlar cambios de esas características. Hecho por un comité de cambios formado por miembros de todas las áreas involucradas en el desarrollo. Permite tener trazabilidad entre IC y saber qué ítems están afectados por un cambio.
- Registrar y reportar los cambios y su estado de implementación.
- Verificar que los requerimientos, documentación y producto construido sean consistentes entre sí.

El objetivo es establecer y mantener la integridad de los productos de software a lo largo de su vida.

Integridad del Producto:

- Satisface las necesidades del usuario.
- Es fácilmente rastreable por completo durante todo su ciclo de vida.
- Satisface los requerimientos de rendimiento.
- Cumple con expectativas de costo.

Problemas Potenciales con los Componentes de Software:

- Pérdida de un componente
- Pérdida de cambios
- Sincronía fuente-objeto-ejecutable
- Regresión de fallas
- Doble mantenimiento
- Superposición de cambios
- Cambios no validados

Ítems de Configuración: todos los artefactos que forman parte del producto o proyecto que pueden sufrir cambios o ser compartidos entre miembros y sobre los cuales necesitamos conocer su estado y evolución.

Ej:

- Documentos de diseño
- Código fuente
- Ejecutables
- Planes
- Requerimientos
- Etc.

Versión: la forma particular de un artefacto en un instante o contexto dado. El control de versiones se refiere a la evolución de un ítem de configuración o varios por separado. Se representan como grafos.

Variante: una versión de un ítem de configuración que evoluciona por separado. Representa una configuración alternativa.

Configuración de Software: un conjunto de ítems de configuración con su correspondiente versión en un momento determinado.

Repositorio: contenedor de ítems de configuración que mantiene la historia de cada ítem con sus atributos y relaciones. Pueden ser una o varias bases de datos. Usado para hacer evaluaciones de impacto de un cambio.

Los usuarios crean y modifican ítems de configuración en directorios locales y luego transfieren al repositorio.

Tipos:

- **Centralizados:** un servidor tiene todos los archivos con sus versiones. Los usuarios traen una parte de los archivos, la modifican y la guardan. Requiere conexión constante y que el servidor esté funcionando.
- **Descentralizados:** cada cliente tiene una copia local completa del repositorio. No requiere conexión constante, solo para sincronizar, y no hay un solo punto de fallo.

Línea Base: una configuración que fue revisada formalmente y sobre la que se llegó a un acuerdo. Sirve como base para desarrollos posteriores y puede cambiarse sólo a través de un procedimiento formal de control de cambios. Permiten retroceder en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto.

Pueden ser:

- De especificación (requerimientos, diseño, etc.)
- De productos que pasaron por un control de calidad definido previamente.

Rama: conjunto de ítems con sus versiones que bifurcan el desarrollo. Hay una rama principal. Las ramas se crean para desarrollar una característica, con razones semánticas o para experimentación. Pueden ser descartadas o integradas.

Integración de Ramas: llevar los cambios de una a otra. Todas las ramas deberían integrarse eventualmente a la principal o descartarse.

Actividades Fundamentales en Detalle:

Identificación IC:

- Identificar de forma única cada IC.
- Convenciones y reglas de nombrado.
- Definir la estructura del repositorio.
- Definir ubicación dentro del repositorio.

Control de Cambios:

- Desarrollo
- Revisión
- Marcar Línea Base (por el comité de control de cambios)
- Propuesta de Cambio (origen del control de cambios)
- Análisis de Impacto
- Revisión de Partes
- Aceptación o Rechazo (por el comité de control de cambios)
- Notificación a Partes

Informes de Estado: mantiene registros de la evolución del sistema. Incluye reportes de rastreabilidad de todos los cambios de las líneas base durante el ciclo de vida.

Auditoría de Configuración: hay dos tipos de auditoría.

- Auditoría Física: asegura que lo que está indicado para cada IC en la línea base o en la actualización se alcanzó realmente.
- Auditoría Funcional: evaluación independiente de los productos controlando que la funcionalidad y el rendimiento reales de cada IC sean consistentes con la especificación de requerimientos.

Sirve para dos propósitos:

- **Validación:** el problema es resuelto de forma correcta.
- **Verificación:** el producto cumple con los objetivos definidos en la documentación de línea base (hace lo correcto). Las funcionalidades se llevan a cabo y los tests pasan o están registrados como problemas reportados.

Plan de Gestión de Configuración: incluye:

- Reglas de nombrado de los IC
- Herramientas a usar para la SCM
- Roles e integrantes del comité de control de cambios
- Procedimiento formal de cambios
- Plantillas de formularios
- Procesos de auditoría